

Competitive Programming Notebook

Programadores Roblox

Contents

1 Geometry	2	4.6 Lis Seg	13
1.1 Inside Polygon	2	4.7 Lis	13
1.2 Point Location	2	4.8 Knapsack	14
1.3 Lattice Points	2	4.9 Bitmask	14
1.4 Convex Hull	2	4.10 Kadane	14
1.5 Point	3	5 Primitives	14
2 Graph	4	6 String	14
2.1 Floyd Warshall	4	6.1 Trie	14
2.2 Acha Pontes	4	6.2 Hashing	14
2.3 Topological Sort	4	6.3 Z Function	14
2.4 Eulerian Path	4	6.4 Kmp	15
2.5 Kruskal	5	7 Search and sort	15
2.6 Kosaraju	5	7.1 Monotonic Stack	15
2.7 Dinitz	5	7.2 Merge Sort	15
2.8 Min Cost Max Flow	6	8 Math	15
2.9 Diametro	6	8.1 Divisores	15
2.10 Lca Jc	6	8.2 Segment Sieve	15
2.11 Edmonds-karp	7	8.3 Fft	15
2.12 Lca	7	8.4 Crivo	16
2.13 Khan	8	8.5 Base Calc	16
2.14 Dijkstra	8	8.6 Discrete Log	16
2.15 Bellman Ford	8	8.7 Exgcd	16
3 DS	8	8.8 Matrix	16
3.1 Dsu	8	8.9 Fexp	17
3.2 Trie Bits	8	8.10 Mod Inverse	17
3.3 Segtree2d	9	8.11 Menor Fator Primo	17
3.4 Maximum Subarray Sum Range	9	8.12 Soma Pg Mod	17
3.5 Min Queue	9	8.13 Combinatorics	17
3.6 Segtree Iterativa	9	8.14 Totient	17
3.7 Bit	10	8.15 Equacao Diofantina	18
3.8 Sparse Table	10	9 Stress	18
3.9 Bit2d	10	9.1 Gen	18
3.10 Ordered Set E Map	10	10 General	18
3.11 Seglazy Iterativa	10	10.1 Mex	18
3.12 Seg Lazy Pa	11	10.2 Debug	18
3.13 Segtree Lazy	11	10.3 Bitwise	18
3.14 Merge Sort Tree	12	10.4 Brute Choose	19
3.15 Psum 2d	12	10.5 Count Permutations	19
4 DP	12	10.6 Struct	19
4.1 Disjoint Blocks	12		
4.2 Digit	13		
4.3 Lis Brunomonteiro	13		
4.4 Edit Distance	13		
4.5 Lcs	13		

1 Geometry

1.1 Inside Polygon

```

1 // Convex O(logn)
2
3 bool insideT(point a, point b, point c, point e){
4     int x = ccw(a, b, e);
5     int y = ccw(b, c, e);
6     int z = ccw(c, a, e);
7     return !((x==1 or y==1 or z==1) and (x==-1 or y==-1 or z
8         ==-1));
9 }
10
11 bool inside(vp &p, point e){ // ccw
12     int l=2, r=(int)p.size()-1;
13     while(l<r){
14         int mid = (l+r)/2;
15         if(ccw(p[0], p[mid], e) == 1)
16             l=mid+1;
17         else{
18             r=mid;
19         }
20     }
21     // bordo
22     // if(r==(int)p.size()-1 and ccw(p[0], p[r], e)==0)
23     // return false;
24     // if(r==2 and ccw(p[0], p[1], e)==0) return false;
25     // if(ccw(p[r], p[r-1], e)==0) return false;
26     return insideT(p[0], p[r-1], p[r], e);
27 }
28
29 // Any O(n)
30
31 int inside(vp &p, point pp){
32     // 1 - inside / 0 - boundary / -1 - outside
33     int n = p.size();
34     for(int i=0; i<n; i++){
35         int j = (i+1)%n;
36         if(line({p[i], p[j]}).inside_seg(pp))
37             return 0;
38     }
39     int inter = 0;
40     for(int i=0; i<n; i++){
41         int j = (i+1)%n;
42         if(p[i].x <= pp.x and pp.x < p[j].x and ccw(p[i], p[j], pp)==1)
43             inter++; // up
44         else if(p[j].x <= pp.x and pp.x < p[i].x and ccw(p[i], p[j], pp)==-1)
45             inter++; // down
46     }
47     if(inter%2==0) return -1; // outside
48     else return 1; // inside
49 }

```

1.2 Point Location

```

1
2 int32_t main(){
3     sws;
4
5     int t; cin >> t;
6
7     while(t--){
8
9         int x1, y1, x2, y2, x3, y3; cin >> x1 >> y1 >> x2 >>
10            y2 >> x3 >> y3;
11
12         int deltax1 = (x1-x2), deltax1 = (y1-y2);
13
14         int compx = (x1-x3), compy = (y1-y3);
15
16         int ans = (deltax1*compy) - (compx*deltax1);
17
18         if(ans == 0){cout << "TOUCH\n"; continue;}
19         if(ans < 0){cout << "RIGHT\n"; continue;}
20         if(ans > 0){cout << "LEFT\n"; continue;}
21     }
22     return 0;
23 }

```

1.3 Lattice Points

```

1 ll gcd(ll a, ll b) {
2     return b == 0 ? a : gcd(b, a % b);
3 }
4 ll area_triangulo(ll x1, ll y1, ll x2, ll y2, ll x3, ll y3) {
5     return abs(x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 -
6         y2));
7 }
8 ll pontos_borda(ll x1, ll y1, ll x2, ll y2) {
9     return gcd(abs(x2 - x1), abs(y2 - y1));
10 }
11
12 int32_t main() {
13     ll x1, y1, x2, y2, x3, y3;
14     cin >> x1 >> y1;
15     cin >> x2 >> y2;
16     cin >> x3 >> y3;
17     ll area = area_triangulo(x1, y1, x2, y2, x3, y3);
18     ll tot_borda = pontos_borda(x1, y1, x2, y2) +
19         pontos_borda(x2, y2, x3, y3) + pontos_borda(x3, y3, x1,
20             y1);
21
22     ll ans = (area - tot_borda) / 2 + 1;
23     cout << ans << endl;
24
25     return 0;
26 }

```

1.4 Convex Hull

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4 #define int long long
5 typedef int cod;
6
7 struct point
8 {
9     cod x,y;
10     point(cod x = 0, cod y = 0): x(x), y(y)
11     {}
12
13     double modulo()
14     {
15         return sqrt(x*x + y*y);
16     }
17
18     point operator+(point o)
19     {
20         return point(x+o.x, y+o.y);
21     }
22     point operator-(point o)
23     {
24         return point(x - o.x, y - o.y);
25     }
26     point operator*(cod t)
27     {
28         return point(x*t, y*t);
29     }
30     point operator/(cod t)
31     {
32         return point(x/t, y/t);
33     }
34
35     cod operator*(point o)
36     {
37         return x*o.x + y*o.y;
38     }
39     cod operator^(point o)
40     {
41         return x*o.y - y * o.x;
42     }
43     bool operator<(point o)
44     {
45         if( x != o.x) return x < o.x;
46         return y < o.y;
47     }
48 };
49
50 int ccw(point p1, point p2, point p3)
51 {
52     cod cross = (p2-p1) ^ (p3-p1);
53     if(cross == 0) return 0;
54 }

```

```

55     else if(cross < 0) return -1;
56     else return 1;
57 }
58
59 vector <point> convex_hull(vector<point> p)
60 {
61     sort(p.begin(), p.end());
62     vector<point> L,U;
63
64     //Lower
65     for(auto pp : p)
66     {
67         while(L.size() >= 2 and ccw(L[L.size()-2], L.back(),
68             pp) == -1)
69         {
70             // Ãr -1 pq eu nÃo quero excluir os colineares
71             L.pop_back();
72         }
73         L.push_back(pp);
74     }
75     reverse(p.begin(), p.end());
76
77     //Upper
78     for(auto pp : p)
79     {
80         while(U.size() >= 2 and ccw(U[U.size()-2], U.back(),
81             pp) == -1)
82         {
83             U.pop_back();
84         }
85         U.push_back(pp);
86     }
87     L.pop_back();
88     L.insert(L.end(), U.begin(), U.end()-1);
89     return L;
90 }
91
92 cod area(vector<point> v)
93 {
94     int ans = 0;
95     int aux = (int)v.size();
96     for(int i = 2; i < aux; i++)
97     {
98         ans += ((v[i].x - v[0].x)*(v[i-1].y - v[0].y))/2;
99     }
100     ans = abs(ans);
101     return ans;
102 }
103
104 int bound(point p1 , point p2)
105 {
106     return __gcd(abs(p1.x-p2.x), abs(p1.y-p2.y));
107 }
108 //teorema de pick [pontos = A - (bound+points)/2 + 1]
109
110 int32_t main()
111 {
112
113     int n;
114     cin >> n;
115
116     vector<point> v(n);
117     for(int i = 0; i < n; i++)
118     {
119         cin >> v[i].x >> v[i].y;
120     }
121
122     vector<point> ch = convex_hull(v);
123
124     cout << ch.size() << '\n';
125     for(auto p : ch) cout << p.x << " " << p.y << "\n";
126
127     return 0;
128 }

```

1.5 Point

```

1 template<class T>
2 struct Point {
3     typedef Point P;
4     typedef Point pt;
5     T x, y;
6     explicit Point(T x=0, T y=0) : x(x), y(y) {}

```

```

7     bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
8     bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
9     P operator+(P p) const { return P(x+p.x, y+p.y); }
10    P operator-(P p) const { return P(x-p.x, y-p.y); }
11    P operator*(T d) const { return P(x*d, y*d); }
12    P operator/(T d) const { return P(x/d, y/d); }
13    T operator^(P p) const { return x*p.y - y*p.x; }
14    T dot(P p) const { return x*p.x + y*p.y; } // ||a|| * ||b||
15    T cross(P p) const { return x*p.y - y*p.x; } // ||a|| * ||b|| * sin(theta)
16    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
17    T dist2() const { return x*x + y*y; }
18    double dist() const { return sqrt((double)dist2()); }
19
20    T orient(P a, P b, P c) {return (b-a)^(c-a);}
21
22    // angle to x-axis in interval [-pi, pi]
23    double angle() const { return atan2(y, x); }
24
25    // angle between v and w in [0, pi]
26    double angle(pt v, pt w) {
27        return acos(clamp(dot(v,w) / abs(v) / abs(w), -1.0,
28            1.0));
29    }
30
31    P unit() const { return *this/dist(); } // makes dist()=1
32    P perp() const { return P(-y, x); } // rotates +90 degrees
33    P normal() const { return perp().unit(); }
34    // returns point rotated 'a' radians ccw around the origin
35    P rotate(double a) const {
36        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a));
37    }
38
39    friend ostream& operator<<(ostream& os, P p) {
40        return os << "(" << p.x << "," << p.y << ")";
41    }
42
43    // inside the disk of diameter [a, b]
44    bool inDisk(P a, P b, P p) {
45        return (a-p).dot(b-p) <= 0;
46    }
47
48    bool onSegment(P a, P b, P p) {
49        return orient(a,b,p) == 0 && inDisk(a,b,p);
50    }
51
52    // check if point is in BAC
53    bool inAngle(P a, P b, P c, P p) {
54        assert(orient(a,b,c) != 0);
55        if (orient(a,b,c) < 0) swap(b,c);
56        return orient(a,b,p) >= 0 && orient(a,c,p) <= 0;
57    }
58
59    // return angle BAC ccw in interval [0, 2pi]
60    double orientedAngle(pt a, pt b, pt c) {
61        if (orient(a,b,c) >= 0)
62            return angle(b-a, c-a);
63        else
64            return 2*M_PI - angle(b-a, c-a);
65    }
66
67    bool properInter(P a, P b, P c, P d, P &ans) {
68        double oa = orient(c,d,a),
69            ob = orient(c,d,b),
70            oc = orient(a,b,c),
71            od = orient(a,b,d);
72
73        // Proper intersection exists iff opposite signs
74        if (oa*ob < 0 && oc*od < 0) {
75            P ans = (a*ob - b*oa) / (ob-oa);
76            return true;
77        }
78        return false;
79    }
80
81    set<P> inters(P a, P b, P c, P d) {
82        P out;
83        if (properInter(a,b,c,d,out)) return {out};
84        set<P> s;
85        if (onSegment(c,d,a)) s.insert(a);
86        if (onSegment(c,d,b)) s.insert(b);

```

```

85         if (onSegment(a,b,c)) s.insert(c);
86         if (onSegment(a,b,d)) s.insert(d);
87         return s;
88     }
89
90     // check if polygon is convex
91     bool isConvex(vector<P> p) {
92         bool hasPos=false, hasNeg=false;
93         for (int i=0, n=p.size(); i<n; i++) {
94             int o = orient(p[i], p[(i+1)%n], p[(i+2)%n]);
95             if (o > 0) hasPos = true;
96             if (o < 0) hasNeg = true;
97         }
98         return !(hasPos && hasNeg);
99     }
100 }
101 };

```

2 Graph

2.1 Floyd Warshall

```

1 // SSP e acha ciclos.
2 // Bom com constraints menores.
3 // O(n^3)
4
5 int dist[501][501];
6
7 void floydWarshall() {
8     for(int k = 0; k < n; k++) {
9         for(int i = 0; i < n; i++) {
10            for(int j = 0; j < n; j++) {
11                dist[i][j] = min(dist[i][j], dist[i][k] +
12                                dist[k][j]);
13            }
14        }
15    }
16
17    void solve() {
18        int m, q;
19        cin >> n >> m >> q;
20        for(int i = 0; i < n; i++) {
21            for(int j = i; j < n; j++) {
22                if(i == j) {
23                    dist[i][j] = dist[j][i] = 0;
24                } else {
25                    dist[i][j] = dist[j][i] = linf;
26                }
27            }
28        }
29        for(int i = 0; i < m; i++) {
30            int u, v, w;
31            cin >> u >> v >> w; u--; v--;
32            dist[u][v] = min(dist[u][v], w);
33            dist[v][u] = min(dist[v][u], w);
34        }
35        floydWarshall();
36        while(q--) {
37            int u, v;
38            cin >> u >> v; u--; v--;
39            if(dist[u][v] == linf) cout << -1 << '\n';
40            else cout << dist[u][v] << '\n';
41        }
42    }
43 }

```

2.2 Acha Pontes

```

1 vector<int> d, low, pai; // d[v] Tempo de descoberta (
2 discovery time)
3 vector<bool> vis;
4 vector<int> pontos_articulacao;
5 vector<pair<int, int>> pontes;
6 int tempo;
7
8 vector<vector<int>> adj;
9
10 void dfs(int u) {
11     vis[u] = true;
12     tempo++;
13     d[u] = low[u] = tempo;
14     int filhos_dfs = 0;
15     for (int v : adj[u]) {
16         if (v == pai[u]) continue;

```

```

16         if (vis[v]) { // back edge
17             low[u] = min(low[u], d[v]);
18         } else {
19             pai[v] = u;
20             filhos_dfs++;
21             dfs(v);
22             low[u] = min(low[u], low[v]);
23             if (pai[u] == -1 && filhos_dfs > 1) {
24                 pontos_articulacao.push_back(u);
25             }
26             if (pai[u] != -1 && low[v] >= d[u]) {
27                 pontos_articulacao.push_back(u);
28             }
29             if (low[v] > d[u]) {
30                 pontes.push_back({min(u, v), max(u, v)});
31             }
32         }
33     }
34 }

```

2.3 Topological Sort

```

1 vector<int> adj[MAXN];
2 vector<int> estado(MAXN); // 0: nao visitado 1: processamento
3 // 2: processado
4 vector<int> ordem;
5 bool temCiclo = false;
6
7 void dfs(int v) {
8     if(estado[v] == 1) {
9         temCiclo = true;
10        return;
11    }
12    if(estado[v] == 2) return;
13    estado[v] = 1;
14    for(auto &nei : adj[v]) {
15        if(estado[v] != 2) dfs(nei);
16    }
17    estado[v] = 2;
18    ordem.push_back(v);
19    return;
20 }

```

2.4 Eulerian Path

```

1 /**
2  * versao que assume: #define int long long
3  *
4  * Retorna um caminho/ciclo euleriano em um grafo (se existir
5  * ).
6  * - g: lista de adjacencia (vector<vector<int>>).
7  * - directed: true se o grafo for dirigido.
8  * - s: vertice inicial.
9  * - e: vertice final (opcional). Se informado, tenta caminho
10  * de s ate e.
11  * - O(Nlog(N))
12  * Retorna vetor com a sequencia de vertices, ou vazio se
13  * impossivel.
14  */
15 vector<int> eulerian_path(const vector<vector<int>>& g, bool
16 directed, int s, int e = -1) {
17     int n = (int)g.size();
18     // copia das adjacencias em multiset para permitir
19     remoção especifica
20     vector<multiset<int>> h(n);
21     vector<int> in_degree(n, 0);
22     vector<int> result;
23     stack<int> st;
24     // preencher h e indegrees
25     for (int u = 0; u < n; ++u) {
26         for (auto v : g[u]) {
27             ++in_degree[v];
28             h[u].emplace(v);
29         }
30     }
31     st.emplace(s);
32     if (e != -1) {
33         int out_s = (int)h[s].size();
34         int out_e = (int)h[e].size();
35         int diff_s = in_degree[s] - out_s;
36         int diff_e = in_degree[e] - out_e;
37         if (diff_s * diff_e != -1) return {}; // impossivel
38     }
39     for (int u = 0; u < n; ++u) {

```

```

35         if (e != -1 && (u == s || u == e)) continue;
36         int out_u = (int)h[u].size();
37         if (in_degree[u] != out_u || (!directed && (in_degree
[u] & 1))) {
38             return {};
39         }
40     }
41     while (!st.empty()) {
42         int u = st.top();
43         if (h[u].empty()) {
44             result.emplace_back(u);
45             st.pop();
46         } else {
47             int v = *h[u].begin();
48             auto it = h[u].find(v);
49             if (it != h[u].end()) h[u].erase(it);
50             --in_degree[v];
51             if (!directed) {
52                 auto it2 = h[v].find(u);
53                 if (it2 != h[v].end()) h[v].erase(it2);
54                 --in_degree[u];
55             }
56             st.emplace(v);
57         }
58     }
59     for (int u = 0; u < n; ++u) {
60         if (in_degree[u] != 0) return {};
61     }
62     reverse(result.begin(), result.end());
63     return result;
64 }

```

2.5 Kruskal

```

1 // Ordena as arestas por peso, insere se ja nao estiver no
  mesmo componente
2 // O(E log E)
3
4 struct Edge {
5     int u, v, w;
6     bool operator <(Edge const & other) {
7         return weight < other.weight;
8     }
9 }
10
11 vector<Edge> kruskal(int n, vector<Edge> edges) {
12     vector<Edge> mst;
13     DSU dsu = DSU(n + 1);
14     sort(edges.begin(), edges.end());
15     for (Edge e : edges) {
16         if (dsu.find(e.u) != dsu.find(e.v)) {
17             mst.push_back(e);
18             dsu.join(e.u, e.v);
19         }
20     }
21     return mst;
22 }

```

2.6 Kosaraju

```

1 bool vis[MAXN];
2 vector<int> order;
3 int component[MAXN];
4 int N, m;
5 vector<int> adj[MAXN], adj_rev[MAXN];
6
7 // dfs no grafo original para obter a ordem (pos-order)
8 void dfs1(int u) {
9     vis[u] = true;
10    for (int v : adj[u]) {
11        if (!vis[v]) {
12            dfs1(v);
13        }
14    }
15    order.push_back(u);
16 }
17
18 // dfs o grafo reverso para encontrar os SCCs
19 void dfs2(int u, int c) {
20     component[u] = c;
21     for (int v : adj_rev[u]) {
22         if (component[v] == -1) {
23             dfs2(v, c);
24         }
25     }
26 }

```

```

25     }
26 }
27
28 int kosaraju() {
29     order.clear();
30     fill(vis + 1, vis + N + 1, false);
31     for (int i = 1; i <= N; i++) {
32         if (!vis[i]) {
33             dfs1(i);
34         }
35     }
36     fill(component + 1, component + N + 1, -1);
37     int c = 0;
38     reverse(order.begin(), order.end());
39     for (int u : order) {
40         if (component[u] == -1) {
41             dfs2(u, c++);
42         }
43     }
44     return c;
45 }

```

2.7 Dinitz

```

1 // Complexidade: O(V^2E)
2
3 struct FlowEdge {
4     int from, to;
5     long long cap, flow = 0;
6     FlowEdge(int from, int to, long long cap) : from(from),
    to(to), cap(cap) {}
7 };
8
9 struct Dinic {
10     const long long flow_inf = 1e18;
11     vector<FlowEdge> edges;
12     vector<vector<int>> adj;
13     int qntV, qntE = 0;
14     int source, sink;
15     vector<int> level, ptr;
16     queue<int> q;
17
18     Dinic(int qntV, int source, int sink) : qntV(qntV),
    source(source), sink(sink) {
19         adj.resize(qntV);
20         level.resize(qntV);
21         ptr.resize(qntV);
22     }
23
24     void add_edge(int from, int to, long long cap) {
25         edges.emplace_back(from, to, cap);
26         edges.emplace_back(to, from, 0);
27         adj[from].push_back(qntE);
28         adj[to].push_back(qntE + 1);
29         qntE += 2;
30     }
31
32     bool bfs() {
33         while (!q.empty()) {
34             int from = q.front();
35             q.pop();
36             for (int id : adj[from]) {
37                 if (edges[id].cap == edges[id].flow)
38                     continue;
39                 if (level[edges[id].to] != -1)
40                     continue;
41                 level[edges[id].to] = level[from] + 1;
42                 q.push(edges[id].to);
43             }
44         }
45         return level[sink] != -1;
46     }
47
48     long long dfs(int from, long long pushed) {
49         if (pushed == 0)
50             return 0;
51         if (from == sink)
52             return pushed;
53         for (int& cid = ptr[from]; cid < (int)adj[from].size
    (); cid++) {
54             int id = adj[from][cid];
55             int to = edges[id].to;
56             if (level[from] + 1 != level[to])
57                 continue;
58             long long tr = dfs(to, min(pushed, edges[id].cap
    - edges[id].flow));

```

```

59         if (tr == 0)
60             continue;
61         edges[id].flow += tr;
62         edges[id ^ 1].flow -= tr;
63         return tr;
64     }
65     return 0;
66 }
67
68 long long max_flow() {
69     long long f = 0;
70     while (true) {
71         fill(level.begin(), level.end(), -1);
72         level[source] = 0;
73         q.push(source);
74         if (!bfs())
75             break;
76         fill(ptr.begin(), ptr.end(), 0);
77         while (long long pushed = dfs(source, flow_inf))
78             f += pushed;
79     }
80     return f;
81 }
82
83 };

```

2.8 Min Cost Max Flow

```

1 // Encontra o menor custo para passar K de fluxo em um grafo
  // com N vertices
2 // Funciona com multiplas arestas para o mesmo par de
  // vertices
3 // Para encontrar o min cost max flow eh so fazer K =
  // infinito
4
5 struct Edge {
6     int from, to, capacity, cost, id;
7 };
8
9 vector<vector<array<int, 2>>> adj;
10 vector<Edge> edges; // arestas pares sao as normais e suas
    // reversas sao as impares
11
12 const int INF = LLONG_MAX;
13
14 void shortest_paths(int n, int v0, vector<int>& dist, vector<
    int>& edge_to) {
15     dist.assign(n, INF);
16     dist[v0] = 0;
17     vector<bool> in_queue(n, false);
18     queue<int> q;
19     q.push(v0);
20     edge_to.assign(n, -1);
21
22     while (!q.empty()) {
23         int u = q.front();
24         q.pop();
25         in_queue[u] = false;
26         for (auto [v, id] : adj[u]) {
27             if (edges[id].capacity > 0 && dist[v] > dist[u] +
                edges[id].cost) {
28                 dist[v] = dist[u] + edges[id].cost;
29                 edge_to[v] = id;
30                 if (!in_queue[v]) {
31                     in_queue[v] = true;
32                     q.push(v);
33                 }
34             }
35         }
36     }
37 }
38
39 void add_edge(int from, int to, int capacity, int cost) {
40     edges.push_back({from, to, capacity, cost, (int)edges.
        size()});
41     edges.push_back({to, from, 0, -cost, (int)edges.size()});
42     // reversa
43 }
44
45 int min_cost_flow(int N, int K, int s, int t) {
46     adj.assign(N, vector<array<int, 2>>());
47
48     for (Edge e : edges) {
49         adj[e.from].push_back({e.to, e.id});

```

```

50
51 int flow = 0;
52 int cost = 0;
53 vector<int> dist, edge_to;
54 while (flow < K) {
55     shortest_paths(N, s, dist, edge_to);
56     if (dist[t] == INF)
57         break;
58
59     // find max flow on that path
60     int f = K - flow;
61     int cur = t;
62     while (cur != s) {
63         f = min(f, edges[edge_to[cur]].capacity);
64         cur = edges[edge_to[cur]].from;
65     }
66
67     // apply flow
68     flow += f;
69     cost += f * dist[t];
70     cur = t;
71     while (cur != s) {
72         int edge = edge_to[cur];
73         int rev_edge = edge ^ 1;
74
75         edges[edge].capacity -= f;
76         edges[rev_edge].capacity += f;
77         cur = edges[edge].from;
78     }
79 }
80
81 if (flow < K)
82     return -1;
83 else
84     return cost;
85 }

```

2.9 Diametro

```

1 /*
2 0(N + M), retorna cara mais distante, len
3 do caminho em arestas e os caras do caminho em um vetor
4 Para pegar o diametro da arvore, chamar duas vezes
5 auto [a, DA, PA] = calc(1);
6 auto [b, len_diametro, caminho_diametro] = calc(a);
7 */
8 auto calc = [&](int S) -> tuple<int, int, vector<int>> {
9     queue<pair<int, int>> q;
10     q.push({S, 0});
11     vector<int> dp(n + 1, -1), pai(n + 1, 0);
12     dp[S] = 0;
13     pai[S] = -1;
14     int longe = S;
15     int dist = 0;
16     while (!q.empty()) {
17         auto [u, d] = q.front();
18         q.pop();
19         if (d > dist) {
20             dist = d;
21             longe = u;
22         }
23         for (int v : adj[u]) {
24             if (dp[v] == -1) {
25                 dp[v] = d + 1;
26                 pai[v] = u;
27                 q.push({v, d + 1});
28             }
29         }
30     }
31     int cur = longe;
32     vector<int> caminho;
33     while (cur != -1) {
34         caminho.push_back(cur);
35         cur = pai[cur];
36     }
37     return {longe, dist, caminho};
38 };

```

2.10 Lca Jc

```

1 const int MAXN = 200005;
2 int N;
3 int LOG;
4

```

```

5 vector<vector<int>> adj;
6 vector<int> profundidade;
7 vector<vector<int>> cima; // cima[v][j] eh o 2^j-esimo
  ancestral de v
8
9 void dfs(int v, int p, int d) {
10     profundidade[v] = d;
11     cima[v][0] = p; // o pai direto eh o 2^0-ésimo ancestral
12     for (int j = 1; j < LOG; j++) {
13         // se o ancestral 2^(j-1) existir, calculamos o 2^j
14         if (cima[v][j - 1] != -1) {
15             cima[v][j] = cima[cima[v][j - 1]][j - 1];
16         } else {
17             cima[v][j] = -1; // nao tem ancestral superior
18         }
19     }
20     for (int nei : adj[v]) {
21         if (nei != p) {
22             dfs(nei, v, d + 1);
23         }
24     }
25 }
26
27 void build(int root) {
28     LOG = ceil(log2(N));
29     profundidade.assign(N + 1, 0);
30     cima.assign(N + 1, vector<int>(LOG, -1));
31     dfs(root, -1, 0);
32 }
33
34 int get_lca(int a, int b) {
35     if (profundidade[a] < profundidade[b]) {
36         swap(a, b);
37     }
38     // sobe 'a' ate a mesma profundidade de 'b'
39     for (int j = LOG - 1; j >= 0; j--) {
40         if (profundidade[a] - (1 << j) >= profundidade[b]) {
41             a = cima[a][j];
42         }
43     }
44     // se 'b' era um ancestral de 'a', então 'a' agora é
    igual a 'b'
45     if (a == b) {
46         return a;
47     }
48     // sobe os dois juntos ate encontrar os filhos do LCA
49     for (int j = LOG - 1; j >= 0; j--) {
50         if (cima[a][j] != -1 && cima[a][j] != cima[b][j]) {
51             a = cima[a][j];
52             b = cima[b][j];
53         }
54     }
55     return cima[a][0];
56 }

```

2.11 Edmonds-karp

```

1 // Edmonds-Karp com scalling O(E log(F))
2
3 int n, m;
4 const int MAXN = 510;
5 vector<vector<int>> capacity(MAXN, vector<int>(MAXN, 0));
6 vector<vector<int>> adj(MAXN);
7
8 int bfs(int s, int t, int scale, vector<int>& parent) {
9     fill(parent.begin(), parent.end(), -1);
10    parent[s] = -2;
11    queue<pair<int, int>> q;
12    q.push({s, LLONG_MAX});
13
14    while (!q.empty()) {
15        int cur = q.front().first;
16        int flow = q.front().second;
17        q.pop();
18
19        for (int next : adj[cur]) {
20            if (parent[next] == -1 && capacity[cur][next] >=
21                scale) {
22                parent[next] = cur;
23                int new_flow = min(flow, capacity[cur][next]);
24                if (next == t)
25                    return new_flow;
26                q.push({next, new_flow});
27            }
28        }
29    }
30    return 0;
31 }

```

```

27     }
28 }
29
30 return 0;
31 }
32
33 int maxflow(int s, int t) {
34     int flow = 0;
35     vector<int> parent(MAXN);
36     int new_flow;
37     int scalling = 1ll << 62;
38
39     while (scalling > 0) {
40         while (new_flow = bfs(s, t, scalling, parent)){
41             if (new_flow == 0) continue;
42             flow += new_flow;
43             int cur = t;
44             while (cur != s) {
45                 int prev = parent[cur];
46                 capacity[prev][cur] -= new_flow;
47                 capacity[cur][prev] += new_flow;
48                 cur = prev;
49             }
50             scalling /= 2;
51         }
52     }
53     return flow;
54 }
55 }

```

2.12 Lca

```

1 // LCA - CP algorithm
2 // preprocessing O(NlogN)
3 // lca O(logN)
4 // Uso: criar LCA com a quantidade de vertices (n) e lista de
    adjacencias (adj)
5 // chamar a funcao preprocess com a raiz da arvore
6
7 struct LCA {
8     int n, l, timer;
9     vector<vector<int>> adj;
10    vector<int> tin, tout;
11    vector<vector<int>> up;
12
13    LCA(int n, const vector<vector<int>>& adj) : n(n), adj(
        adj) {}
14
15    void dfs(int v, int p) {
16        tin[v] = ++timer;
17        up[v][0] = p;
18        for (int i = 1; i <= l; ++i)
19            up[v][i] = up[up[v][i-1]][i-1];
20
21        for (int u : adj[v]) {
22            if (u != p)
23                dfs(u, v);
24        }
25
26        tout[v] = ++timer;
27    }
28
29    bool is_ancestor(int u, int v) {
30        return tin[u] <= tin[v] && tout[u] >= tout[v];
31    }
32
33    int lca(int u, int v) {
34        if (is_ancestor(u, v))
35            return u;
36        if (is_ancestor(v, u))
37            return v;
38        for (int i = l; i >= 0; --i) {
39            if (!is_ancestor(up[u][i], v))
40                u = up[u][i];
41        }
42        return up[u][0];
43    }
44
45    void preprocess(int root) {
46        tin.resize(n);
47        tout.resize(n);
48        timer = 0;
49        l = ceil(log2(n));
50        up.assign(n, vector<int>(l + 1));
51        dfs(root, root);
52    }

```

53 };

2.13 Khan

```

1 // topo-sort DAG
2 // lexicograficamente menor.
3 // N: numero de v rtices (1-indexado)
4 // adj: lista de adjacencia do grafo
5
6 const int MAXN = 5 * 1e5 + 2;
7 vector<int> adj[MAXN];
8 int N;
9
10 vector<int> kahn() {
11     vector<int> indegree(N + 1, 0);
12     for (int u = 1; u <= N; u++) {
13         for (int v : adj[u]) {
14             indegree[v]++;
15         }
16     }
17     priority_queue<int, vector<int>, greater<int>> pq;
18     for (int i = 1; i <= N; i++) {
19         if (indegree[i] == 0) {
20             pq.push(i);
21         }
22     }
23     vector<int> result;
24     while (!pq.empty()) {
25         int u = pq.top();
26         pq.pop();
27         result.push_back(u);
28         for (int v : adj[u]) {
29             indegree[v]--;
30             if (indegree[v] == 0) {
31                 pq.push(v);
32             }
33         }
34     }
35     if (result.size() != N) {
36         return {};
37     }
38     return result;
39 }

```

2.14 Dijkstra

```

1 // SSP com pesos positivos.
2 // O((V + E) log V).
3
4 vector<int> dijkstra(int S) {
5     vector<bool> vis(MAXN, 0);
6     vector<ll> dist(MAXN, LLONG_MAX);
7     dist[S] = 0;
8     priority_queue<pii, vector<pii>, greater<pii>> pq;
9     pq.push({0, S});
10    while(pq.size()) {
11        ll v = pq.top().second;
12        pq.pop();
13        if(vis[v]) continue;
14        vis[v] = 1;
15        for(auto &[peso, vizinho] : adj[v]) {
16            if(dist[vizinho] > dist[v] + peso) {
17                dist[vizinho] = dist[v] + peso;
18                pq.push({dist[vizinho], vizinho});
19            }
20        }
21    }
22    return dist;
23 }

```

2.15 Bellman Ford

```

1 struct Edge {
2     int u, v, w;
3 };
4
5 // se x = -1, nao tem ciclo
6 // se x != -1, pegar pais de x pra formar o ciclo
7
8 int n, m;
9 vector<Edge> edges;
10 vector<int> dist(n);
11 vector<int> pai(n, -1);

```

```

12
13 for (int i = 0; i < n; i++) {
14     x = -1;
15     for (Edge &e : edges) {
16         if (dist[e.u] + e.w < dist[e.v]) {
17             dist[e.v] = max(-INF, dist[e.u] + e.w);
18             pai[e.v] = e.u;
19             x = e.v;
20         }
21     }
22 }
23
24 // achando caminho (se precisar)
25 for (int i = 0; i < n; i++) x = pai[x];
26
27 vector<int> ciclo;
28 for (int v = x;; v = pai[v]) {
29     cycle.push_back(v);
30     if (v == x && ciclo.size() > 1) break;
31 }
32 reverse(ciclo.begin(), ciclo.end());

```

3 DS

3.1 Dsu

```

1 struct DSU {
2     vector<int> parent, size;
3     int C;
4     DSU(int n) : parent(n + 1), size(n + 1), C(n) {
5         for (int i = 1; i <= n; i++) {
6             parent[i] = i;
7             size[i] = 1;
8         }
9     }
10    int find(int a) {
11        if (parent[a] == a) return a;
12        return parent[a] = find(parent[a]);
13    }
14    int same(int a, int b) {
15        return find(a) == find(b);
16    }
17    int get_size(int a) {
18        return size[find(a)];
19    }
20    int count() {
21        return C;
22    }
23    void join(int a, int b) {
24        int pA = find(a);
25        int pB = find(b);
26        if (pA != pB) {
27            C--;
28            if (size[pA] > size[pB]) {
29                swap(pA, pB);
30            }
31            parent[pA] = pB;
32            size[pB] += size[pA];
33        }
34    }
35 };

```

3.2 Trie Bits

```

1 int trie[MAXN][2];
2 int cnt[MAXN][2];
3 int cur_node = 0;
4
5 void insere(int x) {
6     int node = 0;
7     for (int i = 29; ~i; i--) {
8         int bit = (1ll << i) & x ? 1 : 0;
9         if (trie[node][bit] == 0) trie[node][bit] = ++
            cur_node;
10        cnt[node][bit]++;
11        node = trie[node][bit];
12    }
13 }
14
15 int query(int x) {
16     int r = 0;
17     int node = 0;
18     for (int i = 29; ~i; i--) {

```



```

19         int bit = (x >> i) & 1;
20         if (trie[node][bit ^ 1] != 0 && cnt[node][bit ^ 1] >
0) {
21             r |= (1ll << i);
22             node = trie[node][bit ^ 1];
23         } else {
24             node = trie[node][bit];
25         }
26     }
27     return r;
28 }

```

3.3 Segtree2d

```

1 // 0 index
2 struct SegTree2d {
3     int n;
4     vector<vector<int>>> tree;
5     SegTree2d(int n) : n(n) {
6         tree.assign(4 * n, vector<int>(4 * n, 0));
7     }
8     void update_x(int no_y, int no_x, int lx, int rx, int x,
9         int val, bool folha_y) {
10         if (lx == rx) {
11             if (folha_y) tree[no_y][no_x] = val;
12             else tree[no_y][no_x] = tree[2 * no_y][no_x] + tree[2 *
13                 no_y + 1][no_x];
14             return;
15         }
16         int mid = (lx + rx) / 2;
17         if (x <= mid) update_x(no_y, 2 * no_x, lx, mid, x, val,
18             folha_y);
19         else update_x(no_y, 2 * no_x + 1, mid + 1, rx, x, val,
20             folha_y);
21         tree[no_y][no_x] = tree[no_y][2 * no_x] + tree[no_y][2 *
22             no_x + 1];
23     }
24     void update_y(int no_y, int ly, int ry, int y, int x, int
25         val) {
26         if (ly == ry) {
27             update_x(no_y, 1, 0, n - 1, x, val, true);
28             return;
29         }
30         int mid = (ly + ry) / 2;
31         if (y <= mid) update_y(2 * no_y, ly, mid, y, x, val);
32         else update_y(2 * no_y + 1, mid + 1, ry, y, x, val);
33         update_x(no_y, 1, 0, n - 1, x, val, false);
34     }
35     int query_x(int no_y, int no_x, int lx, int rx, int x1, int
36         x2) {
37         if (rx < x1 || lx > x2) return 0;
38         if (lx >= x1 && rx <= x2) return tree[no_y][no_x];
39         int mid = (lx + rx) / 2;
40         return query_x(no_y, 2 * no_x, lx, mid, x1, x2) + query_x
41             (no_y, 2 * no_x + 1, mid + 1, rx, x1, x2);
42     }
43     int query_y(int no_y, int ly, int ry, int y1, int y2, int
44         x1, int x2) {
45         if (ry < y1 || ly > y2) return 0;
46         if (ly >= y1 && ry <= y2) return query_x(no_y, 1, 0, n -
47             1, x1, x2);
48         int mid = (ly + ry) / 2;
49         return query_y(2 * no_y, ly, mid, y1, y2, x1, x2) +
50             query_y(2 * no_y + 1, mid + 1, ry, y1, y2, x1, x2);
51     }
52     void update(int y, int x, int val) {
53         update_y(1, 0, n - 1, y, x, val);
54     }
55     int query(int y1, int x1, int y2, int x2) {
56         return query_y(1, 0, n - 1, y1, y2, x1, x2);
57     }
58 };

```

3.4 Maximum Subarray Sum Range

```

1 struct SegTree {
2     int n;
3     vector<array<int, 4>> tree;
4     vector<int> v;
5     SegTree(vector<int> &a) : v(a), n(a.size()) {
6         tree.resize(4 * n);
7         build(1, 0, n - 1);
8     }
9     void build(int x, int lx, int rx) {

```

```

10         if (lx == rx) {
11             tree[x] = { v[lx], v[lx], v[lx], v[lx] };
12             return;
13         }
14         int mid = lx + (rx - lx) / 2;
15         build(2 * x, lx, mid);
16         build(2 * x + 1, mid + 1, rx);
17         tree[x] = combine(tree[2 * x], tree[2 * x + 1]);
18     }
19     array<int, 4> query(int x, int lx, int rx, int l, int r) {
20         if (lx >= l && rx <= r)
21             return tree[x];
22         if (lx > r || rx < l)
23             return { 0, LINF, LINF, LINF };
24         int mid = lx + (rx - lx) / 2;
25         return combine(query(2 * x, lx, mid, l, r), query(2 * x +
26             1, mid + 1, rx, l, r));
27     };
28     int query(int l, int r) {
29         return max(0ll, query(1, 0, n - 1, l, r)[3]);
30     }
31     void update(int x, int lx, int rx, int pos, int val) {
32         if (lx == rx) {
33             tree[x] = { val, val, val, val };
34             return;
35         }
36         int mid = lx + (rx - lx) / 2;
37         if (pos <= mid)
38             update(2 * x, lx, mid, pos, val);
39         else
40             update(2 * x + 1, mid + 1, rx, pos, val);
41         tree[x] = combine(tree[2 * x], tree[2 * x + 1]);
42     }
43     void update(int pos, int val) {
44         update(1, 0, n - 1, pos, val);
45     }
46     array<int, 4> combine(array<int, 4> x, array<int, 4> y) {
47         return {
48             x[0] + y[0],
49             max(x[1], x[0] + y[1]),
50             max(y[2], y[0] + x[2]),
51             max({x[3], y[3], x[2] + y[1]})
52         };
53     }

```

3.5 Min Queue

```

1 struct MinQueue {
2     deque<pair<int, int>> dq;
3     void push(int val, int idx) {
4         while (!dq.empty() && dq.back().first >= val) dq.
5             pop_back();
6         dq.emplace_back(val, idx);
7     }
8     void pop(int idx) {
9         if (!dq.empty() && dq.front().second == idx) {
10             dq.pop_front();
11         }
12     }
13     int get() {
14         return dq.front().first;
15     }
16     bool empty() {
17         return dq.empty();
18     }
19 };
20 /*
21 considerando janela de tamanho K
22 mq.push(v[i], i);
23 if (i >= k) {
24     mq.pop(i - k);
25 }
26 if (i >= k - 1) {
27     result.push_back(mq.get());
28 }
29 */

```

3.6 Segtree Iterativa

```

1 // Exemplo de uso:
2 // SegTree<int> st(vetor);
3 // range query e point update
4 template <typename T>
5 struct SegTree {

```

```

6   int n;
7   vector<T> tree;
8   T neutral_value = 0;
9   T combine(T a, T b) {
10      return a + b;
11   }
12
13   SegTree(const vector<T>& data) {
14      n = data.size();
15      tree.resize(2 * n, neutral_value);
16
17      for (int i = 0; i < n; i++)
18         tree[n + i] = data[i];
19
20      for (int i = n - 1; i > 0; --i)
21         tree[i] = combine(tree[i * 2], tree[i * 2 + 1]);
22   }
23   T query(int l, int r) {
24      T res_l = neutral_value, res_r = neutral_value;
25
26      for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
27         if (l & 1) res_l = combine(res_l, tree[l++]);
28         if (r & 1) res_r = combine(tree[--r], res_r);
29      }
30
31      return combine(res_l, res_r);
32   }
33   void update(int pos, T new_val) {
34      tree[pos += n] = new_val;
35      for (pos >>= 1; pos > 0; pos >>= 1)
36         tree[pos] = combine(tree[2 * pos], tree[2 * pos +
37      1]);
38   };

```

3.7 Bit

```

1   struct BIT {
2       int n;
3       vector<int> bit;
4       BIT(int n = 0): n(n), bit(n + 1, 0) {}
5       void add(int i, int delta) {
6           for (; i <= n; i += i & -i) bit[i] += delta;
7       }
8       int sum(int i) {
9           int r = 0;
10          for (; i > 0; i -= i & -i) r += bit[i];
11          return r;
12      }
13      int range_sum(int l, int r) {
14          if (r < l) return 0;
15          return sum(r) - sum(l - 1);
16      }
17  };

```

3.8 Sparse Table

```

1   // 0-index, O(1)
2   struct SparseTable {
3       vector<vector<int>> st;
4       int max_log;
5       SparseTable(vector<int>& arr) {
6           int n = arr.size();
7           max_log = floor(log2(n)) + 1;
8           st.resize(n, vector<int>(max_log));
9           for (int i = 0; i < n; i++) {
10              st[i][0] = arr[i];
11          }
12          for (int j = 1; j < max_log; j++) {
13              for (int i = 0; i + (1 << j) <= n; i++) {
14                  st[i][j] = max(st[i][j - 1], st[i + (1 <<
15              - 1)][j - 1]);
16              }
17          }
18      }
19      int query(int L, int R) {
20          int tamanho = R - L + 1;
21          int k = floor(log2(tamanho));
22          return max(st[L][k], st[R - (1 << k) + 1][k]);
23      };

```

3.9 Bit2d

```

1   // 1-index
2   struct BIT2d {
3       int n, m;
4       vector<vector<int>> tree;
5       BIT2d(int n, int m) : n(n), m(m) {
6           tree.assign(n + 1, vector<int>(m + 1, 0));
7       }
8       void add(int y, int x, int delta) {
9           for (int i = y; i <= n; i += i & -i) {
10              for (int j = x; j <= m; j += j & -j) {
11                  tree[i][j] += delta;
12              }
13          }
14      }
15      int pref(int y, int x) {
16          int res = 0;
17          for (int i = y; i > 0; i -= i & -i) {
18              for (int j = x; j > 0; j -= j & -j) {
19                  res += tree[i][j];
20              }
21          }
22          return res;
23      }
24      int query(int y1, int x1, int y2, int x2) {
25          return pref(y2, x2) - pref(y1 - 1, x2) - pref(y2, x1 - 1)
26              + pref(y1 - 1, x1 - 1);
27      };

```

3.10 Ordered Set E Map

```

1
2   #include<ext/pb_ds/assoc_container.hpp>
3   #include<ext/pb_ds/tree_policy.hpp>
4   using namespace __gnu_pbds;
5   using namespace std;
6
7   template<typename T> using ordered_multiset = tree<T,
8       null_type, less_equal<T>, rb_tree_tag,
9       tree_order_statistics_node_update>;
10
11   template<typename T> using o_set = tree<T, null_type, less<T>,
12       rb_tree_tag, tree_order_statistics_node_update>;
13
14   template<typename T, typename R> using o_map = tree<T, R,
15       less<T>, rb_tree_tag, tree_order_statistics_node_update>;
16
17   int main() {
18       int i, j, k, n, m;
19       o_set<int> st;
20       st.insert(1);
21       st.insert(2);
22       cout << *st.find_by_order(0) << endl; // k-esimo elemento
23       // menores que k
24       cout << st.order_of_key(2) << endl; // numero de elementos
25       // menores que k
26       o_map<int, int> mp;
27       mp.insert({1, 10});
28       mp.insert({2, 20});
29       cout << mp.find_by_order(0)->second << endl; // k-esimo
30       // elemento
31       cout << mp.order_of_key(2) << endl; // numero de elementos
32       // (chave) menores que k
33       return 0;
34   }

```

3.11 Seglasy Iterativa

```

1   template<typename T, typename L>
2   struct SegLazy {
3       int n, h;
4       vector<T> tree;
5       vector<L> lazy;
6       vector<bool> has_lazy;
7       vector<int> sz; // se len = pow2, remove e sz[p] = 1 << h
8       [p]
9
10      // --- Mudar aqui ---
11      const T T_neutral = LLONG_MAX;
12
13      inline T combine(T a, T b) {
14          return min(a, b);
15      }
16
17      inline T apply_lazy_to_node(T node, L delta, int len) {
18          return node + (T)delta;
19      }

```

```

18 }
19
20 inline L combine_lazy(L old_delta, L new_delta) {
21     return old_delta + new_delta;
22 }
23 // -----
24
25 SegLazy(const vector<T>& data) {
26     n = data.size();
27     h = 32 - __builtin_clz(n);
28     tree.assign(2 * n, T_neutral);
29     lazy.assign(n, L());
30     has_lazy.assign(n, false);
31     sz.assign(2 * n, 1);
32
33     for (int i = 0; i < n; i++) tree[n + i] = data[i];
34     for (int i = n - 1; i > 0; --i) {
35         tree[i] = combine(tree[i << 1], tree[i << 1 | 1]);
36     }
37     sz[i] = sz[i << 1] + sz[i << 1 | 1];
38 }
39
40 inline void apply(int p, L v) {
41     tree[p] = apply_lazy_to_node(tree[p], v, sz[p]);
42     if (p < n) {
43         if (has_lazy[p]) lazy[p] = combine_lazy(lazy[p],
44 v);
45         else lazy[p] = v;
46         has_lazy[p] = true;
47     }
48 }
49
50 inline void build(int p) {
51     p += n;
52     while (p > 1) {
53         p >>= 1;
54         T res = combine(tree[p << 1], tree[p << 1 | 1]);
55         if (has_lazy[p]) tree[p] = apply_lazy_to_node(res,
56 lazy[p], sz[p]);
57         else tree[p] = res;
58     }
59 }
60
61 inline void push(int p) {
62     p += n;
63     for (int s = h; s > 0; --s) {
64         int i = p >> s;
65         if (has_lazy[i]) {
66             apply(i << 1, lazy[i]);
67             apply(i << 1 | 1, lazy[i]);
68             has_lazy[i] = false;
69         }
70     }
71 }
72
73 void update(int l, int r, L v) {
74     int lo = l, ro = r;
75     push(lo); push(ro);
76     for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
77         if (l & 1) apply(l++, v);
78         if (r & 1) apply(--r, v);
79     }
80     build(lo); build(ro);
81 }
82
83 T query(int l, int r) {
84     push(l); push(r);
85     T res_l = T_neutral, res_r = T_neutral;
86     for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
87         if (l & 1) res_l = combine(res_l, tree[l++]);
88         if (r & 1) res_r = combine(tree[--r], res_r);
89     }
90     return combine(res_l, res_r);
91 }
92
93
94 lazy_a[esq] += lazy_a[x]
95 lazy_d[esq] += lazy_d[x]
96 pro da direita tem que mudar o a (que eh o elemento inicial
97     naquele no da direita)
98 lazy_a[dir] += a + len_esq * (d)
99 lazy_d[dir] += lazy_d[x]
100 */
101
102 int gauss(int n, int a, int d) {
103     // (a0 + an) * n / 2
104     if (n <= 0)
105         return 0;
106     return (a + (a + (n - 1) * d)) * n / 2;
107 }
108
109 struct SegTree {
110     int n;
111     vector<int> v, lazy_a, lazy_d, tree;
112     SegTree(vector<int> &a) : v(a), n(a.size()) {
113         tree.resize(4 * n);
114         lazy_a.resize(4 * n, 0ll);
115         lazy_d.resize(4 * n, 0ll);
116         build(1, 0, n - 1);
117     }
118
119     void build(int x, int lx, int rx) {
120         if (lx == rx) {
121             tree[x] = v[lx];
122             return;
123         }
124         int mid = (lx + rx) / 2;
125         build(2 * x, lx, mid);
126         build(2 * x + 1, mid + 1, rx);
127         tree[x] = tree[2 * x] + tree[2 * x + 1];
128     }
129
130     int query(int x, int lx, int rx, int l, int r) {
131         push(x, lx, rx);
132         if (lx >= l && rx <= r)
133             return tree[x];
134         if (lx > r || rx < l)
135             return 0;
136         int mid = (lx + rx) / 2;
137         return query(2 * x, lx, mid, l, r) + query(2 * x + 1, mid
138 + 1, rx, l, r);
139     }
140
141     int query(int l, int r) {
142         return query(1, 0, n - 1, l, r);
143     }
144
145     void push(int x, int lx, int rx) {
146         if (lazy_a[x] && lazy_d[x]) {
147             int tam = rx - lx + 1;
148             tree[x] += gauss(tam, lazy_a[x], lazy_d[x]);
149             if (lx != rx) {
150                 int mid = (lx + rx) / 2;
151                 int tam_esq = mid - lx + 1;
152                 lazy_a[2 * x] += lazy_a[x];
153                 lazy_d[2 * x] += lazy_d[x];
154                 lazy_a[2 * x + 1] += (lazy_a[x] + tam_esq * lazy_d[x
155 ]);
156                 lazy_d[2 * x + 1] += lazy_d[x];
157             }
158             lazy_a[x] = lazy_d[x] = 0;
159         }
160     }
161
162     void update(int x, int lx, int rx, int l, int r, int a, int
163 d) {
164         push(x, lx, rx);
165         if (lx >= l && rx <= r) {
166             lazy_a[x] += a + (lx - l) * d;
167             lazy_d[x] += d;
168             push(x, lx, rx);
169             return;
170         }
171         if (lx > r || rx < l)
172             return;
173         int mid = (lx + rx) / 2;
174         update(2 * x, lx, mid, l, r, a, d);
175         update(2 * x + 1, mid + 1, rx, l, r, a, d);
176         tree[x] = tree[2 * x] + tree[2 * x + 1];
177     }
178
179     void update(int l, int r, int a, int d) {
180         update(1, 0, n - 1, l, r, a, d);
181     }
182 };

```

3.12 Seg Lazy Pa

```

1 /* Notas
2 PA eh da forma a0 (a) e razão (d)
3 na hora de propagar o lazy
4 aplica no no S = (a0 + (a0 + (n - 1) * d)) * n / 2 (
5     variante da soma total do no)
6 pros filhos, o da esquerda eh normal

```

3.13 Segtree Lazy

```

1  /*
2  Seg com double Lazy de soma e de set
3  tipo 1 - soma em range
4  tipo 2 - set em range
5  */
6  struct SegTree {
7      int n;
8      vector<int> v, tree;
9      vector<int> lazy_soma, lazy_set;
10     SegTree(vector<int> &a) : v(a), n(a.size()) {
11         tree.resize(4 * n);
12         lazy_soma.resize(4 * n, 0);
13         lazy_set.resize(4 * n, -1);
14         build(1, 0, n - 1);
15     };
16     void build(int x, int lx, int rx) {
17         if (lx == rx) {
18             tree[x] = v[lx];
19             return;
20         }
21         int mid = (lx + rx) / 2;
22         build(2 * x, lx, mid);
23         build(2 * x + 1, mid + 1, rx);
24         tree[x] = tree[2 * x] + tree[2 * x + 1];
25     }
26     void seta(int x, int lx, int rx, int val) {
27         tree[x] = val * (rx - lx + 1);
28         lazy_set[x] = val;
29         lazy_soma[x] = 0;
30     }
31     void soma(int x, int lx, int rx, int val) {
32         tree[x] += val * (rx - lx + 1);
33         if (lazy_set[x] != -1) {
34             lazy_set[x] += val;
35         } else {
36             lazy_soma[x] += val;
37         }
38     }
39     void push(int x, int lx, int rx) {
40         int mid = (lx + rx) / 2;
41         if (lazy_set[x] != -1) {
42             if (lx != rx) {
43                 seta(2 * x, lx, mid, lazy_set[x]);
44                 seta(2 * x + 1, mid + 1, rx, lazy_set[x]);
45             }
46             lazy_set[x] = -1;
47         }
48         if (lazy_soma[x] != 0) {
49             if (lx != rx) {
50                 soma(2 * x, lx, mid, lazy_soma[x]);
51                 soma(2 * x + 1, mid + 1, rx, lazy_soma[x]);
52             }
53             lazy_soma[x] = 0;
54         }
55     }
56     void update(int x, int lx, int rx, int l, int r, int val,
57         int type) {
58         push(x, lx, rx);
59         if (lx >= l && rx <= r) {
60             if (type == 1) soma(x, lx, rx, val);
61             else seta(x, lx, rx, val);
62             push(x, lx, rx);
63             return;
64         }
65         if (lx > r || rx < l) return;
66         int mid = (lx + rx) / 2;
67         update(2 * x, lx, mid, l, r, val, type);
68         update(2 * x + 1, mid + 1, rx, l, r, val, type);
69         tree[x] = tree[2 * x] + tree[2 * x + 1];
70     }
71     void update(int l, int r, int val, int type) {
72         update(1, 0, n - 1, l, r, val, type);
73     }
74     int query(int x, int lx, int rx, int l, int r) {
75         push(x, lx, rx);
76         if (lx >= l && rx <= r) return tree[x];
77         if (lx > r || rx < l) return 0;
78         int mid = (lx + rx) / 2;
79         return query(2 * x, lx, mid, l, r) + query(2 * x + 1, mid
80             + 1, rx, l, r);
81     }
82     int query(int l, int r) {
83         return query(1, 0, n - 1, l, r);
84     }
85 };

```

3.14 Merge Sort Tree

```

1  #define all(x) x.begin(), x.end()
2
3  struct MST {
4      int n;
5      vector<vector<int>> tree;
6      MST(vector<int> &a) {
7          n = a.size();
8          tree.resize(4 * n);
9          build(1, 0, n - 1, a);
10     }
11     void build(int x, int lx, int rx, vector<int> &a) {
12         if (lx == rx) {
13             tree[x] = { a[lx] };
14             return;
15         }
16         int mid = lx + (rx - lx) / 2;
17         build(2 * x, lx, mid, a);
18         build(2 * x + 1, mid + 1, rx, a);
19         auto &L = tree[2 * x], &R = tree[2 * x + 1];
20         tree[x].resize(L.size() + R.size());
21         merge(all(L), all(R), tree[x].begin());
22     }
23     int query(int x, int lx, int rx, int l, int r, int val) {
24         if (lx > r || rx < l) return 0;
25         if (lx >= l && rx <= r) {
26             auto &v = tree[x];
27             return lower_bound(all(v), val) - v.begin();
28         }
29         int mid = lx + (rx - lx) / 2;
30         return query(2 * x, lx, mid, l, r, val) + query(2 * x +
31             1, mid + 1, rx, l, r, val);
32     }
33     int query(int l, int r, int val) {
34         if (l > r) return 0;
35         return query(1, 0, n - 1, l, r, val);
36     }
37 };
38 /* mst.query(l, r, l) retorna quantos caras distintos no
39     range
40     map<int, int> last;
41     for (int i = 0; i < n; i++) {
42         if (last.count(a[i])) {
43             esq[i] = last[a[i]];
44         } else {
45             esq[i] = -1;
46         }
47         last[a[i]] = i;
48     }
49     MST mst(esq);
50     jogar vetor de ultima aparicao na seg
51 */

```

3.15 Psum 2d

```

1  vector<vector<int>> psum(h+1, vector<int>(w+1, 0));
2  for (int i=1; i<=h; i++){
3      for (int j=1; j<=w; j++){
4          cin >> psum[i][j];
5          psum[i][j] += psum[i-1][j]+psum[i][j-1]-psum[i-1][j]
6              -1;
7      }
8  }
9  // retorna a psum2d do intervalo inclusivo [(a, b), (c, d)]
10 int retangulo(int a, int b, int c, int d){
11     c = min(c, h), d = min(d, w);
12     a = max(0LL, a-1), b = max(0LL, b-1);
13     return v[c][d]-v[a][d]-v[c][b]+v[a][b];
14 }

```

4 DP

4.1 Disjoint Blocks

```

1  // num max de subarrays disjuntos com soma x usando apenas
2  // prefixo i (ou seja, considerando prefixo a[1..i]).
3  int disjointSumX(vector<int> &a, int x) {
4      int n = a.size();
5      map<int, int> best; // best[pref] = melhor dp visto para
6      esse pref

```

```

6     best[0] = 0;
7     int pref = 0;
8     vector<int> dp(n + 1, 0); // dp[0] = 0
9     for (int i = 1; i <= n; i++) {
10         pref += a[i - 1];
11         dp[i] = dp[i-1];
12         auto it = best.find(pref - x);
13         if (it != best.end()) {
14             dp[i] = max(dp[i], it->second + 1);
15         }
16         best[pref] = max(best[pref], dp[i]);
17     }
18     return dp[n];
19 }

```

4.2 Digit

```

1 vector<int> digits;
2
3 int dp[20][10][2][2];
4
5 int rec(int i, int last, int flag, int started) {
6     if (i == (int)digits.size()) return 1;
7     if (dp[i][last][flag][started] != -1) return dp[i][last][
8         flag][started];
9     int lim;
10    if (flag) lim = 9;
11    else lim = digits[i];
12    int ans = 0;
13    for (int d = 0; d <= lim; d++) {
14        if (started && d == last) continue;
15        int new_flag = flag;
16        int new_started = started;
17        if (d > 0) new_started = 1;
18        if (!flag && d < lim) new_flag = 1;
19        ans += rec(i + 1, d, new_flag, new_started);
20    }
21    return dp[i][last][flag][started] = ans;
22 }

```

4.3 Lis Brunomonteiro

```

1 // Calcula e retorna uma LIS O(n.log(n))
2 template<typename T> vector<T> lis(vector<T>& v) {
3     int n = v.size(), m = -1;
4     vector<T> d(n+1, INF);
5     vector<int> l(n);
6     d[0] = -INF;
7
8     for (int i = 0; i < n; i++) {
9         // Para non-decreasing use upper_bound()
10        int t = lower_bound(d.begin(), d.end(), v[i]) - d.
11            begin();
12        d[t] = v[i], l[i] = t, m = max(m, t);
13    }
14
15    int p = n;
16    vector<T> ret;
17    while (p-- > 0) {
18        if (l[p] == m) {
19            ret.push_back(v[p]);
20            m--;
21        }
22    }
23    reverse(ret.begin(), ret.end());
24
25    return ret;
26 }

```

4.4 Edit Distance

```

1 vector<vector<int>> dp(n+1, vector<int>(m+1, LLONG_MIN));
2 for(int j = 0; j <= m; j++) dp[0][j] = j;
3 for(int i = 0; i <= n; i++) dp[i][0] = i;
4 for(int i = 1; i <= n; i++) {
5     for(int j = 1; j <= m; j++) {
6         if(a[i-1] == b[j-1]) {
7             dp[i][j] = dp[i-1][j-1];
8         } else {
9             dp[i][j] = min({dp[i-1][j] + 1, dp[i][j-1] +
10                 1, dp[i-1][j-1] + 1});
11         }
12     }
13 }
14 cout << dp[n][m];

```

4.5 Lcs

```

1 string s1, s2;
2 int dp[1001][1001];
3 int lcs(int i, int j) {
4     if (i < 0 || j < 0) return 0;
5     if (dp[i][j] != -1) return dp[i][j];
6     if (s1[i] == s2[j]) return dp[i][j] = 1 + lcs(i - 1, j -
7         1);
8     else return dp[i][j] = max(lcs(i - 1, j), lcs(i, j - 1));
9 }

```

4.6 Lis Seg

```

1 // comprime coordenadas pra quando ai <= 1e9
2 vector<int> vals(a.begin(), a.end());
3 sort(vals.begin(), vals.end());
4 vals.erase(unique(vals.begin(), vals.end()), vals.end());
5 auto id = [&](int x) -> int {
6     return lower_bound(vals.begin(), vals.end(), x) - vals.
7         begin();
8 };
9
10 for (int i = 0; i < n; i++) a[i] = id(a[i]);
11 // fim da parte de compr
12 SegTree seg(n);
13 // dp[i]: lis que termina em i
14 vector<int> dp(n, 1); // dp[i] = 1 base case
15 for (int i = 0; i < n; i++) {
16     if (a[i] > 0) dp[i] = seg.query(0, max(0ll, a[i] - 1)) +
17         1;
18     seg.update(a[i], dp[i]);
19 }
20 cout << *max_element(dp.begin(), dp.end()) << '\n';

```

4.7 Lis

```

1 int lis_nlogn(vector<int> &v) {
2     vector<int> lis;
3     lis.push_back(v[0]);
4     for (int i = 1; i < v.size(); i++) {
5         if (v[i] > lis.back()) {
6             // estende a lis
7             lis.push_back(v[i]);
8         } else {
9             // encontra o primeiro elemento em lis que >= v[i]
10            auto it = lower_bound(lis.begin(), lis.end(), v[i]
11                );
12            *it = v[i];
13        }
14    }
15    return lis.size();
16 }
17
18 // lis na tree do problema da sub
19 const int MAXN_TREE = 100001;
20 vector<int> adj[MAXN_TREE];
21 int values[MAXN_TREE];
22 int ans = 0;
23
24 void dfs(int u, int p, vector<int>& tails) {
25     auto it = lower_bound(tails.begin(), tails.end(), values[
26         u]);
27     int prev = -1;
28     bool coloquei = false;
29     if (it == tails.end()) {
30         tails.push_back(values[u]);
31         coloquei = true;
32     } else {
33         prev = *it;
34         *it = values[u];
35     }
36     ans = max(ans, (int)tails.size());
37     for (int v : adj[u]) {
38         if (v != p) {
39             dfs(v, u, tails);
40         }
41     }
42     if (coloquei) {
43         tails.pop_back();
44     }
45     *it = prev;
46 }

```

4.8 Knapsack

```

1 // dp[i][j] => i-esimo item com j-carga sobrando na mochila
2 // O(N * W)
3 vector<vector<int>> dp(N + 1, vector<int>(W + 1, 0));
4 for (int i = 1; i <= N; i++) dp[i][0] = 0;
5 for (int j = 0; j <= W; j++) dp[0][j] = 0;
6 for (int i = 1; i <= N; i++) {
7     for (int j = 0; j <= W; j++) {
8         dp[i][j] = dp[i - 1][j];
9         if (j >= items[i - 1].first) {
10             dp[i][j] = max(dp[i][j], dp[i - 1][j - items[i - 1].first] + items[i - 1].second);
11         }
12     }
13 }
14 cout << dp[N][W] << '\n';

```

4.9 Bitmask

```

1 // dp de intervalos com bitmask
2 int prox(int idx) {
3     return lower_bound(S.begin(), S.end(), array<int, 4>{S[idx][1], 011, 011, 011}) - S.begin();
4 }
5
6 int dp[1002][(int)(111 << 10)];
7
8 int rec(int i, int vis) {
9     if (i == (int)S.size()) {
10         if (__builtin_popcountll(vis) == N) return 0;
11         return LLONG_MIN;
12     }
13     if (dp[i][vis] != -1) return dp[i][vis];
14     int ans = rec(i + 1, vis);
15     ans = max(ans, rec(prox(i), vis | (111 << S[i][3])) + S[i][2]);
16     return dp[i][vis] = ans;
17 }

```

4.10 Kadane

```

1 // O(n) - retorna maximum subarray sum
2 array<int, 3> kadane(vector<int> &a) {
3     int L = -1, R = -1, n = a.size();
4     int soma = 0, ans = -INF;
5     int fim = n - 1;
6     for (int i = n - 1; i >= 0; i--) {
7         if (a[i] > a[i] + soma) {
8             fim = i;
9             soma = a[i];
10        } else {
11            soma += a[i];
12        }
13        if (soma > ans) {
14            R = fim;
15            L = i;
16            ans = soma;
17        }
18    }
19    return { ans, L, R };
20 }

```

5 Primitives

6 String

6.1 Trie

```

1 // Trie por array
2 int trie[MAXN][26];
3 int tot_nos = 0;
4 vector<bool> acaba(MAXN, false);
5 vector<int> contador(MAXN, 0);
6
7 void insere(string s) {
8     int no = 0;
9     for (auto &c : s) {
10         if (trie[no][c - 'a'] == 0) {
11             trie[no][c - 'a'] = ++tot_nos;

```

```

12         }
13         no = trie[no][c - 'a'];
14         contador[no]++;
15     }
16     acaba[no] = true;
17 }
18
19 bool busca(string s) {
20     int no = 0;
21     for (auto &c : s) {
22         if (trie[no][c - 'a'] == 0) {
23             return false;
24         }
25         no = trie[no][c - 'a'];
26     }
27     return acaba[no];
28 }
29
30 int isPref(string s) {
31     int no = 0;
32     for (auto &c : s) {
33         if (trie[no][c - 'a'] == 0) {
34             return -1;
35         }
36         no = trie[no][c - 'a'];
37     }
38     return contador[no];
39 }

```

6.2 Hashing

```

1 // String Hash template
2 // constructor(s) - O(|s|)
3 // query(l, r) - returns the hash of the range [l,r] from
4 // left to right - O(1)
5 // query_inv(l, r) from right to left - O(1)
6 // patrocinado por tiagodfs
7 mt19937 rng(time(nullptr));
8
9 struct Hash {
10     const int X = rng();
11     const int MOD = 1e9+7;
12     int n; string s;
13     vector<int> h, hi, p;
14     Hash() {}
15     Hash(string s): s(s), n(s.size()), h(n), hi(n), p(n) {
16         for (int i=0; i<n; i++) p[i] = (i ? X*p[i-1]:1) % MOD;
17         for (int i=0; i<n; i++)
18             h[i] = (s[i] + (i ? h[i-1]:0) * X) % MOD;
19         for (int i=n-1; i>=0; i--)
20             hi[i] = (s[i] + (i+1<n ? hi[i+1]:0) * X) % MOD;
21     }
22     int query(int l, int r) {
23         int hash = (h[r] - (l ? h[l-1]*p[r-l+1]:0)) % MOD;
24         return hash < 0 ? hash + MOD : hash;
25     }
26     int query_inv(int l, int r) {
27         int hash = (hi[l] - (r+1 < n ? hi[r+1]*p[r-l+1]:0)) % MOD;
28         return hash < 0 ? hash + MOD : hash;
29     }
30 };

```

6.3 Z Function

```

1 vector<int> z_function(string s) {
2     int n = s.size();
3     vector<int> z(n);
4     int l = 0, r = 0;
5     for (int i = 1; i < n; i++) {
6         if (i < r) {
7             z[i] = min(r - i, z[i - l]);
8         }
9         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
10             z[i]++;
11         }
12         if (i + z[i] > r) {
13             l = i;
14             r = i + z[i];
15         }
16     }
17     return z;
18 }

```

6.4 Kmp

```

1 vector<int> kmp(string &s) {
2     int m = s.size();
3     vector<int> lsp(m, 0);
4     for (int i = 1, j = 0; i < m; i++) {
5         while (j > 0 && s[j] != s[i]) j = lsp[j - 1];
6         if (s[i] == s[j]) j++;
7         lsp[i] = j;
8     }
9     return lsp;
10 }

```

7 Search and sort

7.1 Monotonic Stack

```

1 vector<int> find_esq(vector<int> &v, bool maior) {
2     int n = v.size();
3     vector<int> result(n);
4     stack<int> s;
5
6     for (int i = 0; i < n; i++) {
7         while (!s.empty() && (maior ? v[s.top()] <= v[i] : v[
8             s.top()] >= v[i])) {
9             s.pop();
10        }
11        if (s.empty()) {
12            result[i] = -1;
13        } else {
14            result[i] = v[s.top()];
15        }
16        s.push(i);
17    }
18    return result;
19 }
20 vector<int> find_dir(vector<int> &v, bool maior) {
21     int n = v.size();
22     vector<int> result(n);
23     stack<int> s;
24     for (int i = n - 1; i >= 0; i--) {
25         while (!s.empty() && (maior ? v[s.top()] <= v[i] : v[
26             s.top()] >= v[i])) {
27             s.pop();
28        }
29        if (s.empty()) {
30            result[i] = -1;
31        } else {
32            result[i] = v[s.top()];
33        }
34        s.push(i);
35    }
36    return result;

```

7.2 Merge Sort

```

1 int mergeAndCount(vector<int> &v, int l, int m, int r) {
2     int x = m - l + 1;
3     int y = r - m;
4     vector<int> left(x), right(y);
5     for (int i = 0; i < x; i++) left[i] = v[l + i];
6     for (int j = 0; j < y; j++) right[j] = v[m + 1 + j];
7     int i = 0, j = 0, k = l;
8     int swaps = 0;
9     while (i < x && j < y) {
10        if (left[i] <= right[j]) {
11            v[k++] = left[i++];
12        } else {
13            v[k++] = right[j++];
14            swaps += (x - i);
15        }
16    }
17    while (i < x) v[k++] = left[i++];
18    while (j < y) v[k++] = right[j++];
19    return swaps;
20 }
21
22 int mergeSort(vector<int> &v, int l, int r) {
23     int swaps = 0;
24     if (l < r) {

```

```

25         int m = l + (r - l) / 2;
26         swaps += mergeSort(v, l, m);
27         swaps += mergeSort(v, m + 1, r);
28         swaps += mergeAndCount(v, l, m, r);
29     }
30     return swaps;
31 }

```

8 Math

8.1 Divisores

```

1 // Retorna um vetor com os divisores de x
2 // eh preciso ter o crivo implementado
3 // 0(divisores)
4
5 vector<int> divs(int x){
6     vector<int> ans = {1};
7     vector<array<int, 2>> primos; // {primo, expoente}
8
9     while (x > 1) {
10        int p = crivo[x], cnt = 0;
11        while (x % p == 0) cnt++, x /= p;
12        primos.push_back({p, cnt});
13    }
14
15    for (int i=0; i<primos.size(); i++){
16        int cur = 1, len = ans.size();
17
18        for (int j=0; j<primos[i][1]; j++){
19            cur *= primos[i][0];
20            for (int k=0; k<len; k++)
21                ans.push_back(cur*ans[k]);
22        }
23    }
24
25    return ans;
26 }

```

8.2 Segment Sieve

```

1 // Retorna quantos primos tem entre [l, r] (inclusivo)
2 // precisa de um vetor com os primos ate sqrt(r)
3 int seg_sieve(int l, int r){
4     if (l > r) return 0;
5     vector<bool> is_prime(r - l + 1, true);
6     if (l == 1) is_prime[0] = false;
7
8     for (int p : primos){
9         if (p * p > r) break;
10        int start = max(p * p, (l + p - 1) / p * p);
11        for (int j = start; j <= r; j += p){
12            if (j >= l) {
13                is_prime[j - l] = false;
14            }
15        }
16    }
17
18    return accumulate(all(is_prime), 0ll);
19 }

```

8.3 Fft

```

1 // multiplica dois polinomios em O(NlogN)
2
3 using cd = complex<double>;
4 const double PI = acos(-1);
5
6 void fft(vector<cd> &A, bool invert) {
7     int N = size(A);
8
9     for (int i = 1, j = 0; i < N; i++) {
10        int bit = N >> 1;
11        for (; j & bit; bit >>= 1)
12            j ^= bit;
13        j ^= bit;
14
15        if (i < j)
16            swap(A[i], A[j]);
17    }
18
19    for (int len = 2; len <= N; len <= 1) {

```

```

20 double ang = 2 * PI / len * (invert ? -1 : 1);
21 cd wlen(cos(ang), sin(ang));
22 for (int i = 0; i < N; i += len) {
23     cd w(1);
24     for (int j = 0; j < len/2; j++) {
25         cd u = A[i+j], v = A[i+j+len/2] * w;
26         A[i+j] = u + v;
27         A[i+j+len/2] = u-v;
28         w *= wlen;
29     }
30 }
31 }
32
33 if (invert) {
34     for (auto &x : A)
35         x /= N;
36 }
37 }
38
39 vector<int> multiply(vector<int> const& A, vector<int> const&
40     B) {
41     vector<cd> fa(begin(A), end(A)), fb(begin(B), end(B));
42     int N = 1;
43     while (N < size(A) + size(B))
44         N <<= 1;
45     fa.resize(N);
46     fb.resize(N);
47     fft(fa, false);
48     fft(fb, false);
49     for (int i = 0; i < N; i++)
50         fa[i] *= fb[i];
51     fft(fa, true);
52
53     vector<int> result(N);
54     for (int i = 0; i < N; i++)
55         result[i] = round(fa[i].real());
56     return result;
57 }

```

8.4 Crivo

```

1 // O(n*log(log(n)))
2 bool composto[MAX]
3 for(int i = 1; i <= n; i++) {
4     if(composto[i]) continue;
5     for(int j = 2*i; j <= n; j += i)
6         composto[j] = 1;
7 }

```

8.5 Base Calc

```

1 int char_to_val(char c) {
2     if (c >= '0' && c <= '9') return c - '0';
3     else return c - 'A' + 10;
4 }
5
6 char val_to_char(int val) {
7     if (val >= 0 && val <= 9) return val + '0';
8     else return val - 10 + 'A';
9 }
10
11 int to_base_10(string &num, int bfrom) {
12     int result = 0;
13     int pot = 1;
14     for (int i = num.size() - 1; i >= 0; i--) {
15         if (char_to_val(num[i]) >= bfrom) return -1;
16         result += char_to_val(num[i]) * pot;
17         pot *= bfrom;
18     }
19     return result;
20 }
21
22 string from_base_10(int n, int bto) {
23     if (n == 0) return "0";
24     string result = "";
25     while (n > 0) {
26         result += val_to_char(n % bto);
27         n /= bto;
28     }
29     reverse(result.begin(), result.end());
30     return result;
31 }
32

```

```

33 string convert_base(string &num, int bfrom, int bto) {
34     int n_base_10 = to_base_10(num, bfrom);
35     return from_base_10(n_base_10, bto);
36 }

```

8.6 Discrete Log

```

1 // returns minimum x for which a^x = b (mod m), a and m are
  coprime.
2 // if the answer dont need to be greater than some value, the
  vector<int> can be removed
3 int discrete_log(int a, int b, int m) {
4     a %= m, b %= m;
5     int n = sqrt(m) + 1;
6
7     int an = 1;
8     for (int i = 0; i < n; ++i)
9         an = (an * 1ll * a) % m;
10
11     unordered_map<int, vector<int>> vals;
12     for (int q = 0, cur = b; q <= n; ++q) {
13         vals[cur].push_back(q);
14         cur = (cur * 1ll * a) % m;
15     }
16
17     int res = LLONG_MAX;
18
19     for (int p = 1, cur = 1; p <= n; ++p) {
20         cur = (cur * 1ll * an) % m;
21         if (vals.count(cur)) {
22             for (int q: vals[cur]){
23                 int ans = n * p - q;
24                 res = min(res, ans);
25             }
26         }
27     }
28     return res;
29 }

```

8.7 Exgcd

```

1 // O retorno da funcao eh {n, m, g}
2 // e significa que gcd(a, b) = g e
3 // n e m sao inteiros tais que an + bm = g
4 array<ll, 3> exgcd(int a, int b) {
5     if(b == 0) return {1, 0, a};
6     auto [m, n, g] = exgcd(b, a % b);
7     return {n, m - a / b * n, g};
8 }

```

8.8 Matrix

```

1 const int MOD = 1e9 + 7;
2
3 struct Matrix {
4     int N, M;
5     vector<vector<int>> mat;
6     Matrix(int n, int m, bool I = false) : N(n), M(m) {
7         mat.assign(N, vector<int>(M, 0ll));
8         if (I) {
9             for (int i = 0; i < min(N, M); i++) {
10                 mat[i][i] = 1;
11             }
12         }
13     };
14     Matrix operator*(const Matrix &other) {
15         assert(M == other.N);
16         Matrix res(N, other.M);
17         for (int i = 0; i < N; i++) {
18             for (int k = 0; k < M; k++) {
19                 for (int j = 0; j < other.M; j++) {
20                     res.mat[i][j] = (res.mat[i][j] + mat[i][k]
21 ] * other.mat[k][j]) % MOD;
22                 }
23             }
24         }
25         return res;
26     }
27     Matrix exp(Matrix A, int B) {
28         assert(A.N == A.M);
29         Matrix res(A.N, A.M, true);
30         while (B) {
31             if (B & 1) res = res * A;

```



```

31         A = A * A;
32         B >>= 1;
33     }
34     return res;
35 }
36 };

```

8.9 Fexp

```

1 // a^e mod m
2 // O(log n)
3
4 int fexp(int a, int e, int m) {
5     a %= m;
6     int ans = 1;
7     while (e > 0) {
8         if (e & 1) ans = ans*a % m;
9         a = a*a % m;
10        e /= 2;
11    }
12    return ans%m;
13 }

```

8.10 Mod Inverse

```

1 array<int, 2> extended_gcd(int a, int b) {
2     if (b == 0) return {1, 0};
3     auto [x, y] = extended_gcd(b, a % b);
4     return {y, x - (a / b) * y};
5 }
6
7 int mod_inverse(int a, int m) {
8     auto [x, y] = extended_gcd(a, m);
9     return (x % m + m) % m;
10 }

```

8.11 Menor Fator Primo

```

1 const int MAXN = 2e6 + 2;
2 int spf[MAXN];
3 vector<int> primes;
4
5 void sieve() {
6     for (int i = 0; i < MAXN; i++) spf[i] = i;
7     for (int i = 2; i < MAXN; i++) {
8         if (spf[i] == i) {
9             for (int j = i * i; j < MAXN; j += i) {
10                if (spf[j] == j) {
11                    spf[j] = i;
12                }
13            }
14        }
15    }
16    for (int i = 2; i < MAXN; i++) {
17        if (spf[i] == i) {
18            primes.push_back(i);
19        }
20    }
21 }
22
23 map<int, int> factorize(int n) {
24     map<int, int> factors;
25     while (n > 1) {
26         factors[spf[n]]++;
27         n /= spf[n];
28     }
29     return factors;
30 }
31
32 int numero_de_divisores(int n) {
33     if (n == 1) return 1;
34     map<int, int> fatores = fatorar(n);
35     int nod = 1;
36     for (auto &[primo, expoente] : fatores) nod *= (expoente
37         + 1);
38     return nod;
39 }
40 // DEFINE INT LONG LONG
41 int soma_dos_divisores(int n) {
42     if (n == 1) return 1;
43     map<int, int> fatores = fatorar(n);
44     int sod = 1;

```

```

45     for (auto &[primo, expoente] : fatores) {
46         int termo_soma = 1;
47         int potencia_primo = 1;
48         for (int i = 0; i < expoente; i++) {
49             potencia_primo *= primo;
50             termo_soma += potencia_primo;
51         }
52         sod *= termo_soma;
53     }
54     return sod;
55 }
56 }

```

8.12 Soma Pg Mod

```

1 // 1 + r + r^2 + r^3 + ... r^n (mod m)
2 // O(log(N)) funciona com qualquer m
3 // retorna {soma, r^n mod m}
4 array<int, 2> soma_pg(int r, int n, int m) {
5     if (n == 0) return {0, 1};
6
7     auto [mid, p] = soma_pg(r, n / 2, m);
8
9     int sum = (mid*p % m + mid) % m;
10    int np = p*p % m; // r^n
11
12    if (n&1) {
13        sum = (sum*r + 1) % m;
14        np = np*r % m;
15    }
16
17    return {sum, np};
18 }

```

8.13 Combinatorics

```

1 const int N = 200005;
2 const int MOD = 1e9 + 7;
3 // DEFINE INT LONG LONG PLMDS
4 int fat[N], fati[N];
5
6 // (a^b) % m em O(log b)
7 // coloque o fexp
8
9 int inv(int n) { return fexp(n, MOD - 2); }
10
11 void precalc() {
12     fat[0] = 1;
13     fati[0] = 1;
14     for (int i = 1; i < N; i++) fat[i] = (fat[i - 1] * i) %
15         MOD;
16     fati[N - 1] = inv(fat[N - 1]);
17     for (int i = N - 2; i >= 0; i--) fati[i] = (fati[i + 1] *
18         (i + 1)) % MOD;
19 }
20
21 int choose(int n, int k) {
22     if (k < 0 || k > n) return 0;
23     return ((fat[n] * fati[k]) % MOD) * fati[n - k] % MOD;
24 }
25
26 // n! / (n-k)!
27 int perm(int n, int k) {
28     if (k < 0 || k > n) return 0;
29     return (fat[n] * fati[n - k]) % MOD;
30 }
31
32 // C_n = (1 / (n+1)) * C(2n, n)
33 int catalan(int n) {
34     if (n < 0 || 2 * n >= N) return 0;
35     int c2n_n = choose(2 * n, n);
36     return (c2n_n * inv(n + 1)) % MOD;
37 }

```

8.14 Totient

```

1 // phi(n) = n * (1 - 1/p1) * (1 - 1/p2) * ...
2 int phi(int n) {
3     int result = n;
4     for (int i = 2; i * i <= n; i++) {
5         if (n % i == 0) {
6             while (n % i == 0)

```

```

7         n /= i;
8         result -= result / i;
9     }
10 }
11 if (n > 1) // SE n sobrou, ele Ãr um fator primo
12     result -= result / n;
13 return result;
14 }
15
16 // crivo phi
17 const int MAXN_PHI = 1000001;
18 int phiv[MAXN_PHI];
19 void phi_sieve() {
20     for (int i = 0; i < MAXN_PHI; i++) phiv[i] = i;
21     for (int i = 2; i < MAXN_PHI; i++) {
22         if (phiv[i] == i) {
23             for (int j = i; j < MAXN_PHI; j += i) phiv[j] -=
24                 phiv[j] / i;
25         }
26     }
27 }

```

8.15 Equacao Diofantina

```

1 int extended_gcd(int a, int b, int& x, int& y) {
2     if (a == 0) {
3         x = 0;
4         y = 1;
5         return b;
6     }
7     int x1, y1;
8     int gcd = extended_gcd(b % a, a, x1, y1);
9     x = y1 - (b / a) * x1;
10    y = x1;
11    return gcd;
12 }
13
14 bool solve(int a, int b, int c, int& x0, int& y0) {
15     int x, y;
16     int g = extended_gcd(abs(a), abs(b), x, y);
17     if (c % g != 0) {
18         return false;
19     }
20     x0 = x * (c / g);
21     y0 = y * (c / g);
22     if (a < 0) x0 = -x0;
23     if (b < 0) y0 = -y0;
24     return true;
25 }

```

9 Stress

9.1 Gen

```

1 // pre-compile os headers:
2 // compilar template com g++ -H e procurar onde esta stdc++.h
3 // criar a pasta bits e incluir com "" ou inves de <>
4 // compilar stress com: g++ -pipe -O3 -flto -march=native -
5 // mtune=native gen.cpp
6 // faz bastante diferenÃa no runtime
7
8 #include <bits/stdc++.h>
9 #include <cstdlib>
10 #include <ctime>
11 using namespace std;
12
13 int randi(int L, int R) { return L + rand() % (R - L + 1); }
14 char randc(char L, char R) { return char(L + rand() % (R - L
15 + 1)); }
16
17 int main(int argc, char** argv) {
18     if (argc > 1) srand(atoi(argv[1]));
19     else srand(time(0));
20
21     int n = randi(1, 100);
22     cout << n << '\n';
23     for (int i = 0; i < n; i++) {
24         cout << randi(-10, 20) << ' ';
25     }
26     cout << '\n';
27 }
28 /*

```

```

28 script.sh
29
30 g++ -std=c++17 -O2 -o sol sol.cpp
31 g++ -std=c++17 -O2 -o brute brute.cpp
32 g++ -std=c++17 -O2 -o gen gen.cpp
33
34 for ((i = 1; ; i++)); do
35     ./gen $i > in.txt
36     ./sol < in.txt > out1.txt
37     ./brute < in.txt > out2.txt
38
39     if ! diff -q out1.txt out2.txt > /dev/null; then
40         echo "Mismatch found on test $i"
41         cat in.txt
42         echo "Your output:"
43         cat out1.txt
44         echo "Brute output:"
45         cat out2.txt
46         break
47     fi
48     echo "Test $i passed"
49 done
50
51 */

```

10 General

10.1 Mex

```

1 struct MEX {
2     map<int, int> f;
3     set<int> falta;
4     int tam;
5     MEX(int n) : tam(n) {
6         for (int i = 0; i <= n; i++) falta.insert(i);
7     }
8     void add(int x) {
9         f[x]++;
10        if (f[x] == 1 && x >= 0 && x <= tam) {
11            falta.erase(x);
12        }
13    }
14    void rem(int x) {
15        if (f.count(x) && f[x] > 0) {
16            f[x]--;
17            if (f[x] == 0 && x >= 0 && x <= tam) {
18                falta.insert(x);
19            }
20        }
21    }
22    int get() {
23        if (falta.empty()) return tam + 1;
24        return *falta.begin();
25    }
26 };

```

10.2 Debug

```

1 template<typename T, typename U>
2 ostream& operator<<(ostream& os, const pair<T, U>& p) {
3     return os << "(" << p.first << ", " << p.second << ")";
4 }
5
6 template<typename T, typename = decltype(begin(declval<T>()))>
7 , typename = enable_if_t<!is_same_v<T, string>>>
8 ostream& operator<<(ostream& os, const T& v) {
9     os << "{";
10    string sep;
11    for (const auto& x : v) os << sep << x, sep = ", ";
12    return os << "}";
13 }
14
15 #define db(x...) cout << "[" << #x << "]: ", [(auto... $){
16     ((cout << $ << "; ", ...); }(x), cout << '\n'

```

10.3 Bitwise

```

1 int check_kth_bit(int x, int k) {
2     return (x >> k) & 1;
3 }
4
5 void print_on_bits(int x) {

```

```

6   for (int k = 0; k < 32; k++) {
7       if (check_kth_bit(x, k)) {
8           cout << k << ' ';
9       }
10  }
11  cout << '\n';
12 }
13
14 int count_on_bits(int x) {
15     int ans = 0;
16     for (int k = 0; k < 32; k++) {
17         if (check_kth_bit(x, k)) {
18             ans++;
19         }
20     }
21     return ans;
22 }
23
24 bool is_even(int x) {
25     return ((x & 1) == 0);
26 }
27
28 int set_kth_bit(int x, int k) {
29     return x | (1 << k);
30 }
31
32 int unset_kth_bit(int x, int k) {
33     return x & ~(1 << k);
34 }
35
36 int toggle_kth_bit(int x, int k) {
37     return x ^ (1 << k);
38 }
39
40 bool check_power_of_2(int x) {
41     return count_on_bits(x) == 1;
42 }

```

10.4 Brute Choose

```

1  vector<int> elements;
2  int N, K;
3  vector<int> comb;
4
5  void brute_choose(int i) {
6      if (comb.size() == K) {
7          for (int j = 0; j < comb.size(); j++) {
8              cout << comb[j] << ' ';
9          }
10         cout << '\n';
11         return;
12     }
13     if (i == N) return;
14     int r = N - i;
15     int preciso = K - comb.size();
16     if (r < preciso) return;
17     comb.push_back(elements[i]);

```

```

18     brute_choose(i + 1);
19     comb.pop_back();
20     brute_choose(i + 1);
21 }

```

10.5 Count Permutations

```

1  // Returns the number of distinct permutations
2  // that are lexicographically less than the string t
3  // using the provided frequency (freq) of the characters
4  // 0(n*freq.size())
5  int countPermLess(vector<int> freq, const string &t) {
6      int n = t.size();
7      int ans = 0;
8
9      vector<int> fact(n + 1, 1), invfact(n + 1, 1);
10     for (int i = 1; i <= n; i++)
11         fact[i] = (fact[i - 1] * i) % MOD;
12     invfact[n] = fexp(fact[n], MOD - 2, MOD);
13     for (int i = n - 1; i >= 0; i--)
14         invfact[i] = (invfact[i + 1] * (i + 1)) % MOD;
15
16     // For each position in t, try placing a letter smaller
17     // than t[i] that is in freq
18     for (int i = 0; i < n; i++) {
19         for (char c = 'a'; c < t[i]; c++) {
20             if (freq[c - 'a'] > 0) {
21                 freq[c - 'a']--;
22                 int ways = fact[n - i - 1];
23                 for (int f : freq)
24                     ways = (ways * invfact[f]) % MOD;
25                 ans = (ans + ways) % MOD;
26                 freq[c - 'a']++;
27             }
28             if (freq[t[i] - 'a'] == 0) break;
29             freq[t[i] - 'a']--;
30         }
31     }
32     return ans;
33 }

```

10.6 Struct

```

1  struct Pessoa{
2      // Atributos
3      string nome;
4      int idade;
5
6      // Comparador
7      bool operator <(const Pessoa& other) const{
8          if(idade != other.idade) return idade > other.idade;
9          else return nome > other.nome;
10     }
11 }

```